COSC419/519:DEEP LEARNING

Optimization and Regularization

Dr. Mohamed Shehata

Computer Science

University of British Columbia

UBC OKANAGAN

Optimization

Optimization

- We have the model represented as a function $f(w, x)$, and a cost function $g(W) = \frac{1}{P}\sum_{i=1}^{P} L(f(W, x_i),\ \ y_i)$ (sometimes also denoted as $J(W)$) to measure the performance of the model
  - $Where\ L(f(W, x_i),\ \ y_i)$ is the loss between current prediction and the ground truth for an input data point i
  - How do we find the weights W that give us the best performance? We **optimize** the weights with respect to the cost function.
- That is, we find the set of weights W^* that minimizes the cost function.

$$W^* = \underset{W}{\operatorname{argmin}}\ g(W)$$

- How can we do this?
  - First idea: randomly search the space of all W values.
    - Bad idea, since it could take an indefinite amount of time, and we are not guaranteed any accuracy improvements.
  - Another idea: start randomly, then follow the direction of "**steepest descent**" of the cost function curve.

Terminology

- **Learning rate:** also known as the step size – controls the magnitude of each step taken by the optimizer to reach the minimum.
- **Batch size:** how many samples are used for each step of the optimizer.
- **Epoch:** one complete pass through the entire training dataset.

Example: let's say dataset size = 200, batch size = 5, epochs = 100, and learning rate = 0.01.

Then dataset is divided into $\frac{200}{5} = 40$ batches, each with 5 training examples. The weights W are updated after every **batch**. Since an epoch is one pass over all training examples, each epoch involves 40 batches. Since there are 100 epochs, it means we go through each batches 100 times, which is a total of 4000 batches or training steps taken by the optimizer. The learning rate of 0.01 says that each step should be 0.01 times

Gradient Descent

- Recall: the "gradient" of a function $f: R^n \rightarrow R$ at a point p is a **vector** in R^n containing the partial derivatives of f with respect to each input variable.
- In other words, the gradient is an n-dimensional vector that tells us the **direction** and **rate of fastest <u>increase</u>**.
- **Key idea**: pick a random starting point in the parameter space of W, then repeatedly take steps in the direction of the **negative** of the gradient.
  - Why **negative** of the gradient?
    - The gradient tells you which way is uphill but you want to get to the valley (minimum), so you walk in the opposite direction (downhill)
    - In simple terms, we assign **random values** for all the weights, then iteratively update them by following the direction of steepest descent (downhill).

Gradient Descent

- How to update the weights?
  - **Step 1**: Calculate the local gradient of the cost function

$$\nabla J(W) = \frac{1}{N}\sum_{i=1}^{N} \nabla J^{(i)}(W)$$

Where N=batch size

i.e. we sum the gradients of the loss from each training example i *(a single datapoint). M*ore on how to compute gradient for each training example later. Then we calculate the average.

  - **Step 2**: Update the parameters W

$$W^{(t+1)} = W^{(t)} - \alpha\, \nabla \mathrm{J}\left(\mathrm{W}^{(t)}\right),$$

where $W^{(t)}$ represents the weights at time t, and α is a constant called the **learning rate**.

Example

- Example-1: J(w) = $w_1^2 + w_2^2$ has a minimum at $(w_1, w_2) = (0,0)$
  - $\nabla J(\boldsymbol{w}) = 0$ can be solved easily.

$$\nabla J(\boldsymbol{w}) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix} = 0 \Rightarrow 2w_1 + 2w_2 = 0 \Rightarrow [w_1, w_2] = [0,0]$$

- Example-2: $J(w) = \frac{1}{e^{w_1 w_2}} + w_1^2 - \sin(w_1 + w_2)$ has a minimum at $(w_1, w_2) = (-0.38, -4.4)$
  - $J(\boldsymbol{w}) = 0$ cannot be solved easily, solved numerically.
  - How? Using gradient descent.

Algorithm

1. Initialize k=0, and pick $\boldsymbol{w}^0$ as a starting point (e.g. use random initialization)
2. Find ∇J($\boldsymbol{w}^k$) (which is $\frac{\partial J}{\partial w^k}$)
3. Update $\boldsymbol{w}^{k+1}$=$\boldsymbol{w}^k$−α $\nabla J(\boldsymbol{w}^k)$ (for some step size α)
4. Increment k
5. Repeat 2-4 until a stopping criterion is reached (convergence or # iterations)

Parameter α controls the step size and is called the learning rate. The larger it is, the faster w changes. We'll see later how to tune the learning rate, but values are typically small, e.g. 0.01 or 0.0001

$$\nabla J(w) = \frac{\partial J}{\partial w} = \begin{pmatrix} \frac{\partial J}{\partial w_1} \\ \vdots \\ \frac{\partial J}{\partial w_D} \end{pmatrix}$$

Algorithm

```
#Vanilla Gradient Descent
w = initialize_weights()
for t in range(1, num_iterations):
    dW = compute_gradients(loss_fn_J, data, w)
    W -= learning_rate * dW
```

Hyperparameters:

- Method of weight initialization
- Number of steps
- Learning rate

Question

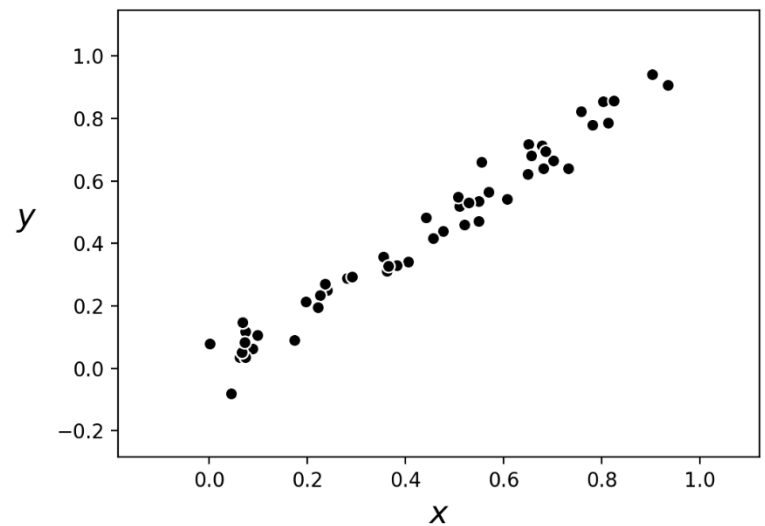
- How is subtracting from a value from the current weights using gradient descent ($\boldsymbol{w}^k - \alpha\ \nabla J(\boldsymbol{w}^k)$) will give us better weights and hence better predictions. While we stated in softmax regression that if we subtract a fixed vector ψ from each weight ($w' = \{w_0 - \psi, \ldots w_{C-1} - \psi\}$), then the predictions should not change?

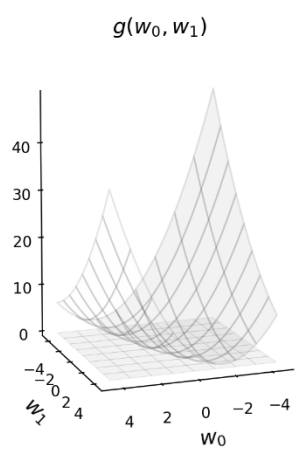
SOLUTION

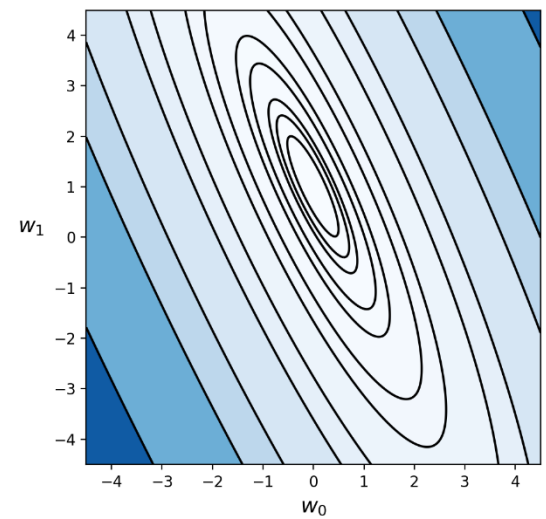
- Gradient Descent: We subtract different values from different weights
  - Each weight gets its own specific gradient value subtracted
- Softmax Invariance: We subtract the same vector from all class weights simultaneously
  - Every class weight vector is shifted by the identical amount

Gradient Descent Example

- In this example we plot the contour and surface plot for the Least Squares cost function for linear regression for an example dataset.
- The contour plot and corresponding surface generated by the Least Squares cost function using this data are shown below.

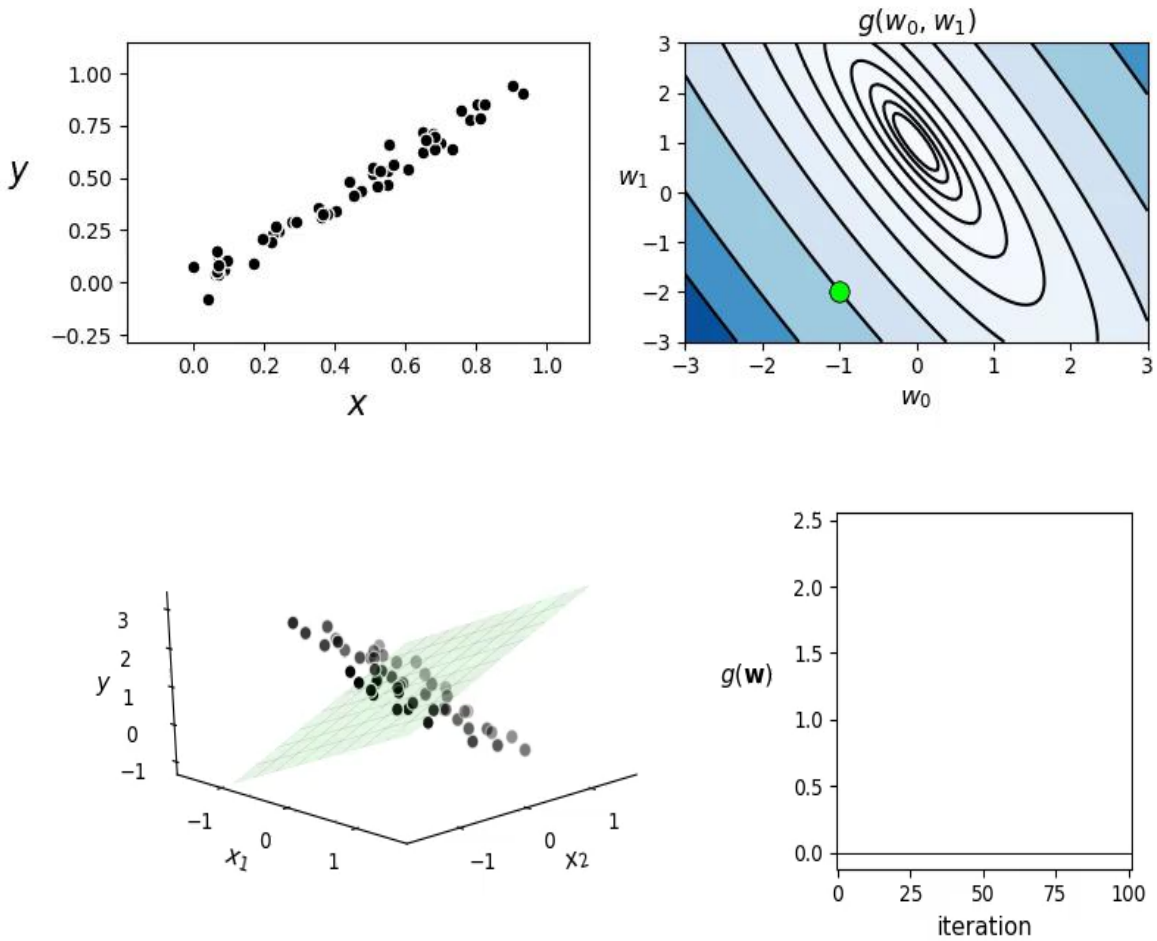






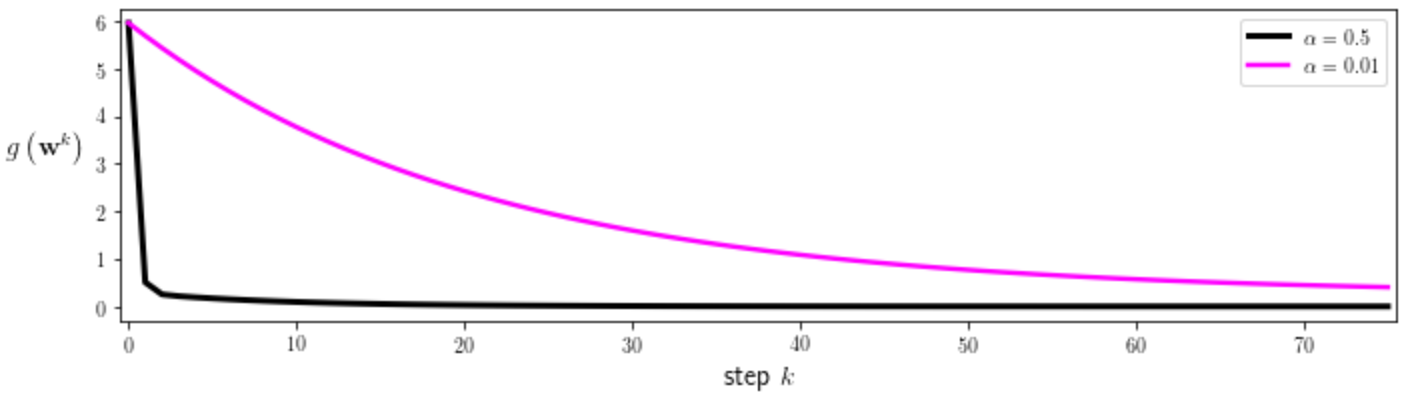
Visual View

- Below we minimize the Least Squares cost using the toy dataset.
- More specifically we use gradient descent and employ a fixed step length value α=0.5 for all 75 steps until approximately reaching the minimum of the function



Effect of α

- We re-create those runs using α=0.5, α=0.01
- Below shows the cost function history plot for each step length value choice. We can see from the plot that indeed the first step length value works considerably better.

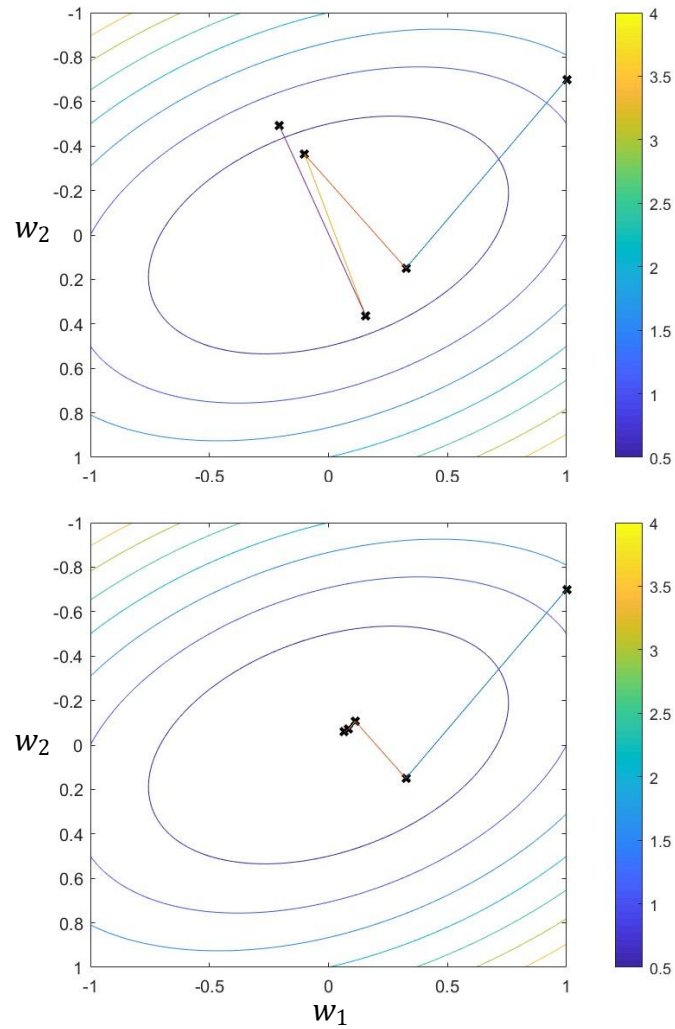


- In this example, we can say α =0.5 is better. The cost function drops dramatically within just the first few steps, reaching near-optimal values very quickly
- However, that this does not mean larger learning rates are always better. If α is too large, gradient descent can overshoot the minimum, oscillate, or even diverge

Gradient Descent Step Size

How do we choose the step size α?

- Small step size $\Rightarrow$ slow convergence
- Large step size $\Rightarrow$ possible fast convergence, or oscillation, or even divergence (top figure)!
- Best chosen to induce the most rapid minimization possible
- Two common methods:
  - Fixed alpha for all iterations
  - Diminishing step size $\alpha = \frac{1}{k}$ at iteration k



Fixed vs. variable step size.

Gradient Descent Convergence Criteria

- We stop the algorithm when there is 'no progress':
  - Either the magnitude of the gradient is too small, i.e., $\left\|\nabla g\left(\boldsymbol{w}^{k-1}\right)\right\|_2 \approx 0$
  - Or the average change in $\boldsymbol{w}$ is too small, i.e., $\frac{1}{\mathrm{N}}\left\|\boldsymbol{w}^{k}-\boldsymbol{w}^{k-1}\right\|_2 \approx 0$
  - Or when evaluations don't differ substantially, i.e., $\left|g\left(\boldsymbol{w}^{k}\right)-g\left(\boldsymbol{w}^{k-1}\right)\right| \approx 0$
  - Or maximum number of iterations is reached
- Recall: Even when such criteria are satisfied, the algorithm may be trapped in a local minimum
- Gradient descent may zig-zag or slowly crawl towards the minimum (local or global)

Full Batch Gradient Descent

Let's talk about the training example data

```
#Vanilla Gradient Descent
w = initialize_weights()
for t in range(1, num_iterations):
    dW = compute_gradients(loss_fn_J, data, w)
    W -= learning_rate * dW
```

Full_data_set

Hyperparameters:

- Method of weight initialization
- Number of steps
- Learning rate

- In **full batch gradient descent** (sometimes called "batch gradient descent"), you use the **entire dataset** to compute the gradient in each iteration.
  - Gradient = average over entire dataset (requires summing over **all** training examples)
  - Most accurate gradient estimate
  - Slowest per iteration (especially for large datasets)
  - This is very impractical, especially if you have N = millions of training examples and your batch size becomes large.
  - W is updated after a full run over the entire dataset (full batch)

Variants of Gradient Descent

- Update the parameters W based on a subset of the training examples (e.g. even **one** training example).
- An optimization algorithm with one training example per iteration is called a **stochastic Gradient Descent** or online algorithm.
  - Gradient = computed from a single training sample
  - Fastest per iteration
  - Noisiest gradient estimate
- Typically, in deep learning we use more than one but less than all the training examples per iteration. In other words, we use **batches** of the training set (called mini-batches stochastic descent).
  - Gradient = average over a small batch
  - Good balance between speed and accuracy
  - Most commonly used in practice
- If not specified clearly, many people use **(SGD)** to refer to the mini-batches stochastic descent. In this course, we follow this convention if not clearly stated otherwise.

Stochastic Gradient Descent

- We refer here to mini-batches stochastic descent
- SGD can be intuitively thought about as an expectation over the full data distribution:

$$L(w) = \mathbb{E}_{(x,y)\sim P_{data}}[L(x,y;w)]$$
$$\cong \frac{1}{m}\sum_{i}^{m} L(x_i, y_i, w)$$

m =size of mini-batch

So the gradient becomes:

$$\nabla_w L(w) = \nabla_w \mathbb{E}_{(x,y)\sim P_{data}}[L(x,y,w)]$$
$$\cong \frac{1}{m}\sum_{i}^{m} \nabla_w L(x_i, y_i, w)$$

- The law of large numbers tells us that the sample mean of m (size of mini-batch) examples approximates the expectation over the entire dataset of N training examples, which is called the approximate expectation via sampling.

SGD Algorithm

- We refer here to mini-batches stochastic descent

```
W = initialize_weights()
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(loss_fn, minibatch, W)
    W -= learning_rate * dW
```

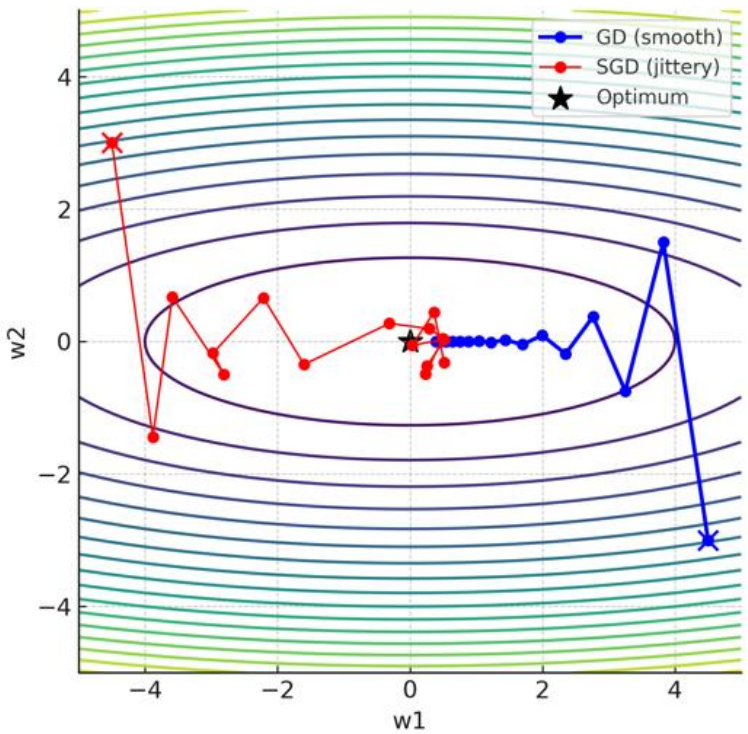
Hyperparameters:

- Method of weight initialization
- Number of steps
- Learning rate
  - In practice, sometimes the learning rate is decayed over iterations
- Data sampling
  - In practice, you randomly sample minibatches or iterate through the data in shuffled order
  - Common approaches include using data loaders (e.g., PyTorch's DataLoader) that handle batching and shuffling automatically
- Batch size
  - Larger batch size means convergence in fewer steps because the updates are less noisy
  - Smaller batch size results in more updates per second since less computation
  - Typical values are 32, 64, 128, 256 – usually a good idea to look at the model's validation accuracy vs training time when tuning batch size
  - It is a good idea to review learning curves of the model validation error against training time with different batch sizes

Problems with SGD

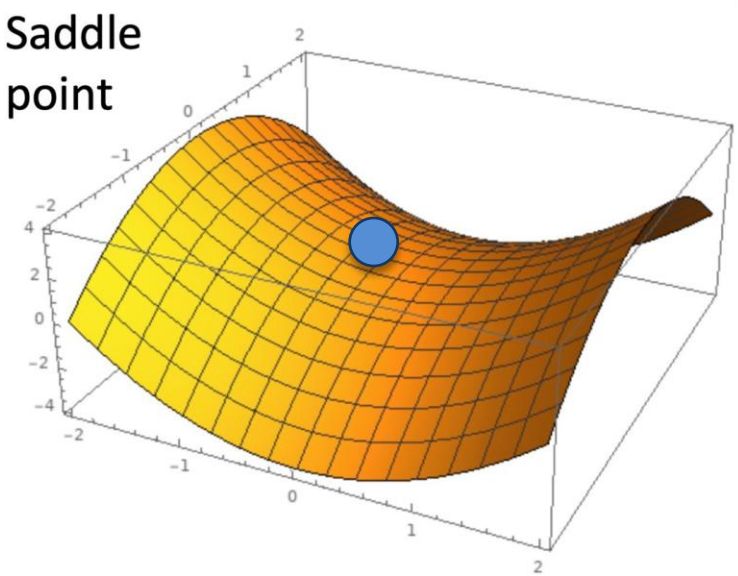
- Suppose our cost function is steep vertically and shallow horizontally. What happens to the updates?
  - Updates have a large vertical component and a shallow horizontal component (also a concern with full-batch GD).

- This cost function has what's called **poor conditioning** optimization landscape:
  - **Steep in w2** (vertical): Large gradients cause aggressive updates
  - **Shallow in w1** (horizontal): Small gradients cause slow progress
- In other words, the updates are smaller in flat directions and very large in directions with steep gradients, resulting in jitter.



Concerns with SGD

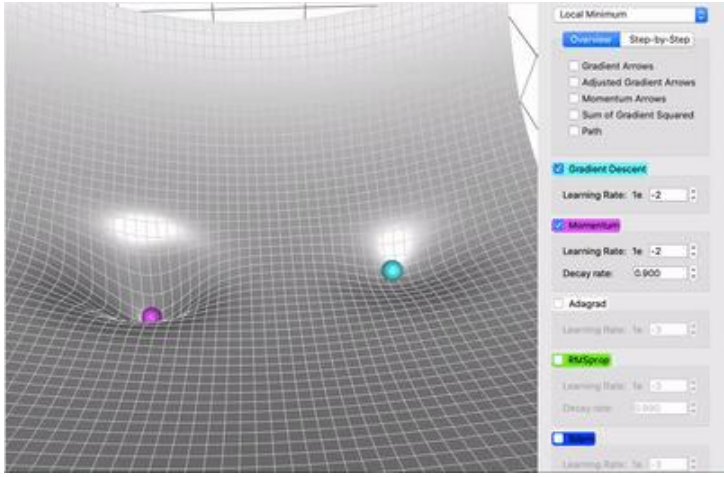
- What if the loss functions runs into a local minima or a saddle point?
- The gradient becomes zero and the weights no longer update.



SGD + Momentum

Key idea: imagine we have a heavy metal ball rolling on a surface.

- The surface represents the cost function, and the ball represents the weight updates
- The ball starts rolling by following the direction of steepest descent (the gradient), gaining some momentum.
- Once it has gained velocity, the direction in which it rolls is not only affected by the direction of steepest descent, but **also** by the momentum it has gained.
- The momentum tries to keep the ball rolling in the direction it was moving in historically.



https://paperswithcode.com/method/sgd-with-momentum
Blue=SGD
Purple=SGD+Momentum

Momentum helps in the following ways:

Damps the oscillation in directions with high curvature

- When the gradient keeps changing direction, momentum acts like a damper. It averages out these opposing movements. The accumulated momentum counteracts sudden direction changes, smoothing out the zigzag path.
- The heavy ball does not instantly change direction when it hits a steep wall. Its momentum carries it forward more steadily instead of bouncing erratically

Builds speed in directions with gentler gradients.

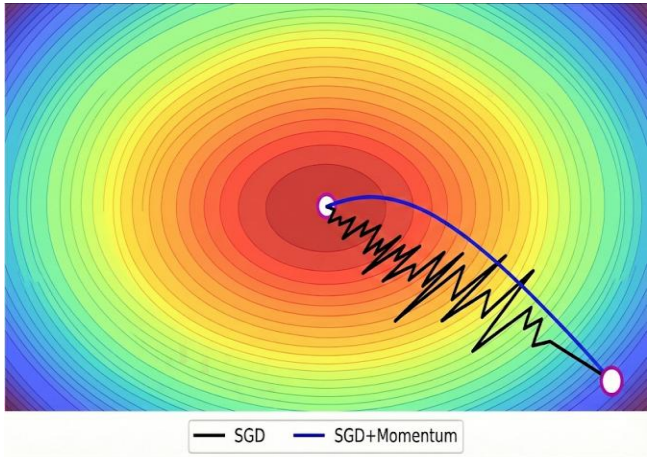
- When gradients consistently point in the same direction (even if small), momentum accumulates over time, building up speed. Each iteration adds a bit more velocity in that direction
- The heavy ball might start rolling slowly on a gentle slope, but it keeps accelerating as momentum builds up, eventually moving much faster than a single gradient step would allow.

SGD + Momentum

Recall the standard SGD algorithm:

$$w_{t+1} = w_t - \alpha \nabla L(w_t)$$

```
for t in range(1, num_iterations):
 minibatch = get_sample_data(data, batch_size)
 dW = compute_gradients(W)
 W -= learning_rate * dW
```



Then this is SGD with momentum:

$$v_{t+1} = \rho v_t + \nabla L(w_t)$$

$$w_{t+1} = w_t - \alpha v_{t+1}$$

- Update weights using velocity
- The direction is based on both the accumulated velocity and the current gradient

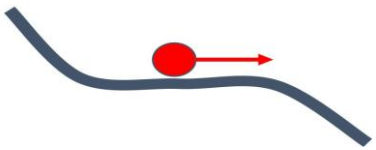
- v_t = velocity (momentum), keeps tracks of historical values of gradient, initialized to 0. Builds up "velocity" with gradients as weighted moving average of past gradients
- ρv_t = keep portion of previous velocity, ρ acts like a friction (typically ρ = 0.9 or 0.99)
- $\alpha \nabla L(w_t)$ = add current gradient

```
v = 0
for t in range(1, num_iterations):
   minibatch = get_sample_data(data, batch_size)
   dW = compute_gradients(W)
   v = rho*v + dW
   W -= learning_rate * v
```

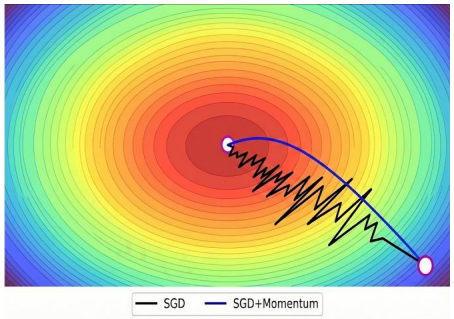
Problems that SGD + Momentum Can help Solve:

(1) Local Minima

(2) Saddle Points



(3) Poor Conditioning



Advantages of SGD with Momentum

- Generally. SGD+Momentum is faster than stochastic gradient descent – hence, the training will be faster than SGD.
- Momentum can help escape shallow local minima because accumulated velocity can "carry" the optimization through small bumps

AdaGrad

- Short for **Adaptive Gradient**, AdaGrad is an optimization algorithm that it has a similar idea to momentum, but instead we keep track of historical **squared sum** of gradients.

```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += dW * dW
    W -= learning_rate * dW / (sqrt(grad_squared) + epsilon)
```

Note: we add `epsilon=1e-7` to prevent division by 0

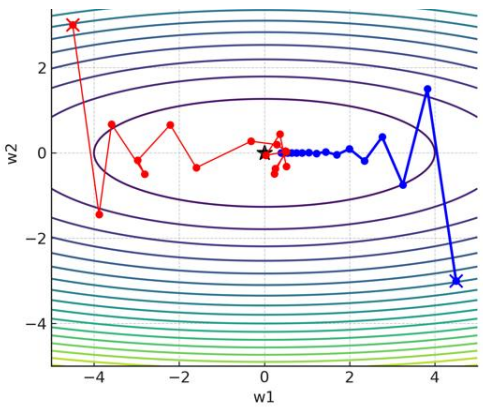
- Dividing the update value by the square root of grad_squared improves learning by **adapting** the gradient updates for each weight based on how it was updated historically.

AdaGrad

```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += dW * dW
    W -= learning_rate * dW / (sqrt(grad_squared) + epsilon)
```

Weights Update:

- Remember the example from before when the updates had a large vertical component and a shallow horizontal component
  - Vertical direction (w_2):
    - Large, oscillating gradients
    - grad_squared accumulates to a large value
    - Divide by large sqrt(grad_squared) → dampens these updates
    - Reduces wild oscillations
  - Horizontal direction (w_1):
    - Small, consistent gradients
    - grad_squared stays relatively small
    - Divide by small sqrt(grad_squared) → accelerates these updates



AdaGrad

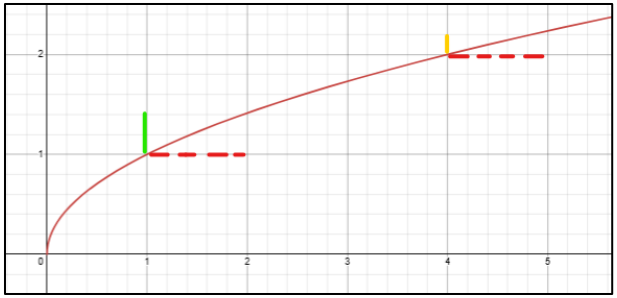
```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += dW * dW
    W -= learning_rate * dW / (sqrt(grad_squared) + epsilon)
```

The Square Root Effect:

- The graph shows why we use sqrt instead of just dividing by grad_squared:
  - sqrt(1) to sqrt(2): difference = 0.414 (larger gap)
  - sqrt(4) to sqrt(5): difference = 0.22 (smaller gap)
  - Why this matters: The square root function grows slower for larger values, so the dampening effect is less severe. This prevents over-dampening parameters that already have large accumulated gradients.

The Adaptivity

- The algorithm is called adaptive because:
  - Each weight gets its own effective learning rate
  - The learning rate automatically adjusts based on the parameter gradient history
  - No manual tuning needed for different weights



Difference between sqrt(1) and sqrt(2) = 0.414
Difference between sqrt(4) and sqrt(5) = 0.22

Problem with AdaGrad

What happens when AdaGrad runs for many iterations?

```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += dW * dW
    W -= learning_rate * dW / (sqrt(grad_squared) + epsilon)
```

- grad_squared will grow to be very large.
- The overall weight update adjustments will keep growing, until the updates are so large that the cost function stops converging, before we reach the optimal weights.

RMSProp (Leaky AdaGrad)

- Stands for **Root Mean Squared Propagation**
- Unlike regular AdaGrad where all past gradients accumulate forever, RMSProp allows old gradient information to gradually "leak away" over time

```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += dW * dW
    W -= learning_rate * dW / (sqrt(grad_squared) + epsilon)
```

AdaGrad

```
grad_squared = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    grad_squared += decay_rate * grad_squared + (1 - decay_rate) * dW * dW
    W -= learning_rate * dW / (np.sqrt(grad_squared) + epsilon)
```

RMSProp

- It uses a **decay rate** to decay the running average of squared gradients. Can think of it as ‘friction’.
  - **If decay_rates= 0,** Information from iteration 1 is still fully present at iteration 1000 (traditional AdaGrad)
  - **If decay_rates= 0.9,** Information from many iterations ago gets multiplied by 0.9 × 0.9 × 0.9... many times.
    - **Eventually becomes negligibly small - it has "leaked out"**

Adam Optimizer (RMSProp+Momentum)

- The Adam optimization algorithm (derived from 'adaptive moment estimation') combines the ideas of momentum and RMSProp, and works extremely well in practice

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    minibatch = get_sample_data(data, batch_size)
    dW = compute_gradients(W)
    first_moment = beta1 * first_moment + (1 - beta1) * dW
    second_moment = beta2 * second_moment + (1 - beta2) * dW * dW
    W -= learning_rate * first_moment / (sqrt(second_moment) + epsilon)
```

Momentum

RMSProp

Naïve implementation, directly combine RMSProp with momentum

- However, what happens at the beginning, when the second moment is initialized to 0 and say beta2 is set to 0.999?
  - The first time we compute second_moment, it will be a value close to 0, and in the update rule, we also divide by the square root of this small number.
  - This means we will have initially have very large updates while the second moment is initially close to 0. This means very large first step.
  - This is not a major issue for first_moment because we multiply by it (it is in the numerator)

Adam Optimizer

- Solution: add a **bias correction** term, to account for the fact that the first and second moment start at 0 in the first time step.

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dW = compute_gradients(W)
    first_moment = beta1 * first_moment + (1 - beta1) * dW
    second_moment = beta2 * second_moment + (1 - beta2) * dW * dW
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    W -= learning_rate * first_unbias / (sqrt(second_unbias) + epsilon)
```

Momentum

RMSProp

Bias correction

- Introduced in 2014 by Kingma et. al in a paper titled "Adam: A Method for Stochastic Optimization".
  - Has over 200k citations!
  - Kingama and Ba, "Adam: A Method for Stochastic Optimization", https://arxiv.org/abs/1412.6980

Using Adam with beta1 = 0.9, beta2 = 0.999, and learning_rate = 1e-3 or 5e-4 is a great starting point for many models

Example:

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
    dW = compute_gradients(W)
    first_moment = beta1 * first_moment + (1 - beta1) * dW
    second_moment = beta2 * second_moment + (1 - beta2) * dW * dW
    first_unbias = first_moment / (1 - beta1 ** t)
    second_unbias = second_moment / (1 - beta2 ** t)
    W -= learning_rate * first_unbias / (sqrt(second_unbias) + epsilon)
```

beta1 = 0.9, beta2 = 0.999, also assume constant gradient dW = 1.0 and learning_rate = 0.001

# Iteration t=1
# Without Bias correction
first_moment = 0.9 * 0 + 0.1 * 1.0 = 0.1
second_moment = 0.999 * 0 + 0.001 * 1.0 = 0.001
step_size = 0.001 * 0.1 / sqrt(0.001 + 1e-7) = 0.001 * 0.1 / 0.0316 = 0.001 * 3.16 ≈ 0.00316 (3x larger)

#With Bias correction
first_moment = 0.1
second_moment = 0.001
first_unbias = 0.1 / (1 - 0.9^1) = 0.1 / 0.1 = 1.0
second_unbias = 0.001 / (1 - 0.999^1) = 0.001 / 0.001 = 1.0
step_size = 0.001 * 1.0 / sqrt(1.0 + 1e-7) ≈ 0.001 (good step)

Iteration t=10
#Without Bias correction
first_moment ≈ 0.651
second_moment ≈ 0.00995
step_size ≈ 0.001 * 0.651 / sqrt(0.00995 + + 1e-7) ≈ 0.00653 # After 10 iterations, moments have accumulated more

#With Bias correction
first_unbias = 0.651 / (1 - 0.9^10) ≈ 1.0
second_unbias = 0.00995 / (1 - 0.999^10) ≈ 1.0
step_size ≈ 0.001 * 1.0 / sqrt(1.0+ + 1e-7) ≈ 0.0009999999 ≈ 0.001 # After 10 iterations, step size is still good

Optimization Algorithms Comparison

Algorithm	Tracks first moments	Tracks second moments (Adaptive rates)	Leaky second moments	Bias correction for moments
SGD	✗	✗	✗	✗
SGD+Momentum	✓	✗	✗	✗
AdaGrad	✗	✓	✗	✗
RMSProp	✗	✓	✓	✗
Adam	✓	✓	✓	✓

Evaluating a Model

Model Evaluation

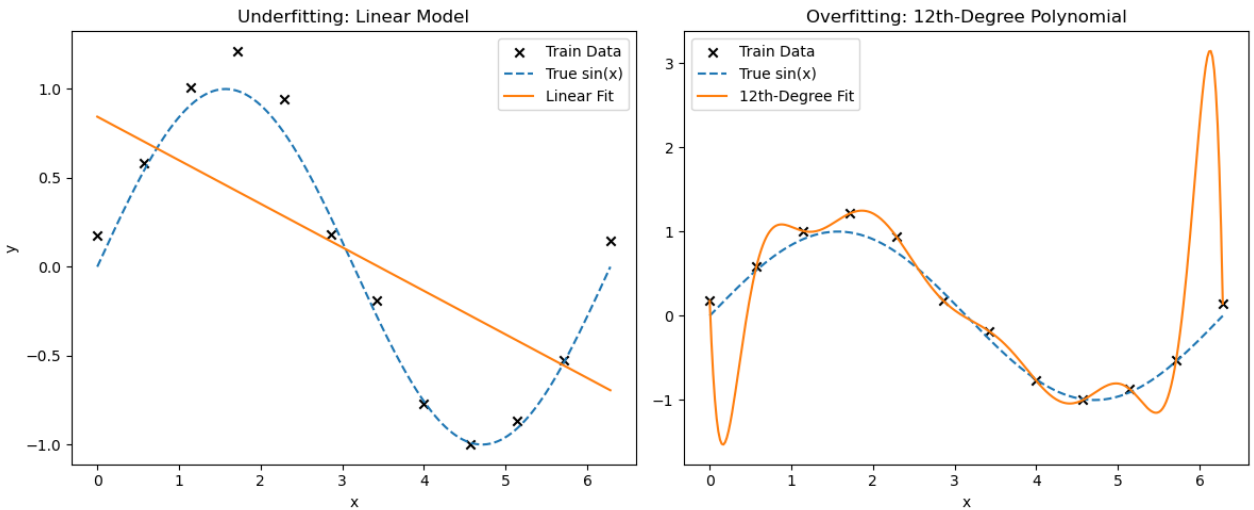
- We can train a model using any optimization algorithm to minimize the cost function over the training data, but how do we know how good the model will perform in the real world?
  - How do we measure performance in the “real world”? We can **test** the model against examples that it has **not seen** during training, and see how it performs on those.
- It is often infeasible to acquire new data just for testing a model, so the standard practice is to **split** the dataset we already have into a **training set** and **test set**.
  - In traditional ML, the training set usually accounts for 60-80% of the dataset. However, in the era of deep learning and big data, datasets can consist of millions of training examples, and so the training set can be upwards of 90% of the dataset

Model Evaluation

- We usually measure model performance using accuracy, but we can also use other metrics such as precision, recall, or many similar metrics depending on the application.
- However, whichever metric we choose, we measure it on both the training set and the test set. We can then compare these two metrics to see how our model is performing:
  - If the training accuracy and test accuracy are roughly the same, then it means our model generalizes well
  - If there is a large gap between the training accuracy and the test accuracy, then our model is not generalizing well, and has adapted too much to the training data.

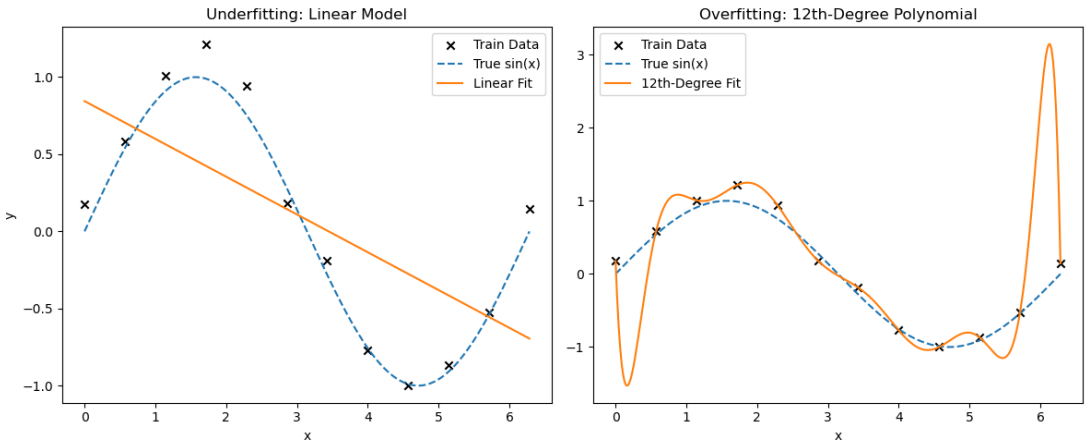
Model Evaluation

- Imagine we train two models to fit the same training data, one is a simple linear model and the other is a complex, high-order polynomial. Below are the two models plotted after optimization:



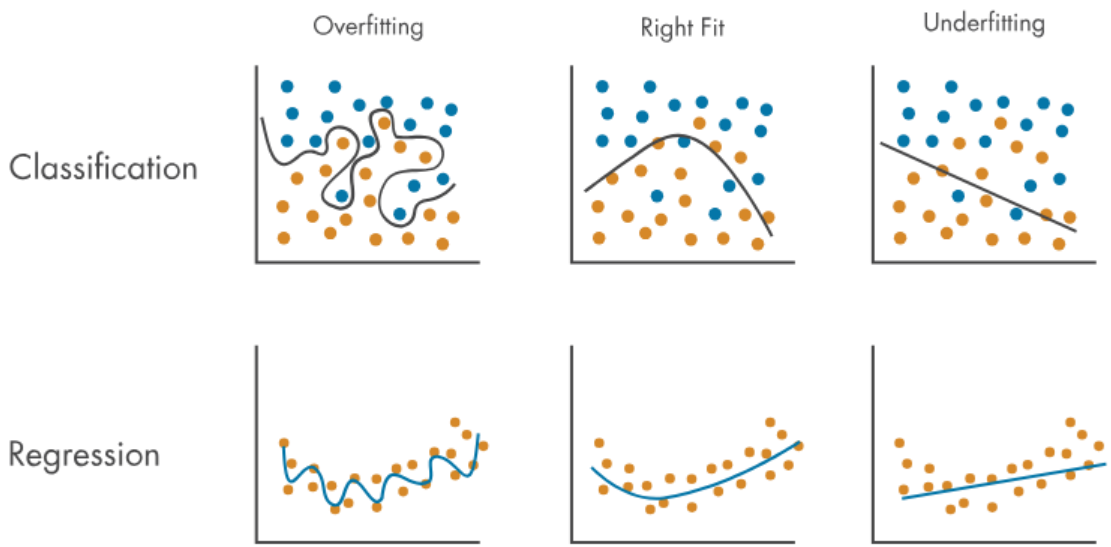
- $f_{left} = w_0 + w_1 x, and\ f_{right} = w_0 + w_1 x + w_2 x^2 + \cdots + w_{12} x^{12}$. Model on the right is more complex because it has more parameters
- Clearly, we see that the linear model is not accurate at all. It is missing the target consistently.
- The 12th-degree polynomial does fit to the training points, but it almost fits **too well**. It would not perform well on unseen data.

Bias and Variance



- **Bias** = How far off your model's *average* predictions are from the true values
  - Caused by: Model is too simple to capture the underlying pattern
  - Result: Underfitting - poor performance on both training and test data
  - Example: Using a straight line to fit a sine wave (like the left plot on previous slide)
- **Variance** = How much your model predictions *change* if you use it on different datasets
  - Caused by: Model is too complex and “memorizes” noise/outliers
  - Result: Overfitting - great on training data, terrible on new test data
  - Example: 12th-degree polynomial with wild oscillations (like the right plot on previous slide)

- The bias-variance tradeoff is a fundamental concept in machine learning



- There is always noise and outlies
- You need to be able to generalize well
- You need to prevent overfitting
- We need to account for unseen (test) data, the goal is to have good performance while testing, not training
- We need some mechanism that we can add to the loss function to discourage the model to help with these issues
- We can add a penalty term to the loss function
- This is called **Regularization**

Regularization

Properties of the Weights

- Assume we have found the perfect W with Loss = 0. Is this weight matrix W unique?

No, because, for example, using 2W will also result in Loss=0. Let's look at an example. Here are the unnormalized outputs using W:

cracker	**7.2**	1.3	2.2
Mug	5.1	**4.9**	2.5
Sugar	-1.7	2.0	**4.1**
Loss	0	0	0

$$loss = g(w_0, \dots, w_{c-1}) = -\frac{1}{P}\sum_{p=1}^{P}\left(\max_{c=0,\dots,C-1} f_{w_c}(x_p) - f_{w_{y_p}}(x_p)\right)$$

Now use L'= 2L

New Loss L' = -1/3 [(max(14.4, 10.2, - 3.4) – 14.2) + (max(2.6, 9.8, 4.0) – 9.8) + (max(2.2, 2.5, 8.2) – 8.2)]

= -1/3 [14.2 – 14.2 + 9.8 – 9.8 + 8.2 – 8.2]

= 0 = L

Note: here we use the multi-class perceptron cost function instead of cross entropy for simplicity, but the same concept applies

Regularization

- So we want to minimize the loss while also forcing the model to "behave well". We can achieve this with **regularization**.

$$g(W) = \frac{1}{P}\sum_{i=1}^{P} L(f_i(W, x_i), y_i) + \lambda R(W)$$

Data Loss: average loss across training examples, i.e. mismatch between model predictions and ground truth

Regularization Term: forces model to "behave well" (see next slide)

- $f(W, x_i)$: **model output** for training example x_i, denoted below as f_i
- $L(f_i, y_i)$: **loss** computed for the i-th training example
- $\mathrm{R}(W)$: **regularization function** – this is a function of the weights, whose value is always positive. Example functions we could use are shown in next slides.
- λ: **regularization parameter** – the 'amount' of regularization. The higher the value, the more we force the model to behave a certain way.
  - This value forces the magnitudes of the columns of W to "behave well", for instance, by preventing any one set of weights w_c growing rapidly while others stay close to zero, therefore preventing overreliance on any specific w_c
  - This is a constant value chosen manually, usually set to 10^{-3} or less.

Scenarios Where Regularization Is Needed

- There are several common scenarios where regularization can be useful:
  - High Feature Count: When you have many features (variables) in your dataset, there's a risk of overfitting. Regularization helps control the complexity of your model and prevents it from becoming too sensitive to the specifics of the training data.
  - Small Dataset: If your dataset is small, it's easier for a model to overfit the limited data. Regularization can help balance the model's fit to the training data and its ability to generalize to new, unseen examples.
  - Noisy Data: Data with a lot of noise can cause models to capture the noise instead of the true signal. Regularization helps by keeping the model's complexity in check, preventing it from memorizing the noise.
  - High Model Complexity: Highly complex models, such as deep neural networks, can easily overfit the training data. Regularization techniques like dropout and weight decay can help these models generalize better.

https://medium.com/

Regularization Functions: L2

The main functions commonly used for regularization are **L1 regularization** and **L2 regularization**:

- *L2 Regularization:* $R(W) = \sum_{i,j} W_{ij}^2$
  - It is the sum of the squares of all the weights
  - **Penalizes large weights:** since the weights are squared, bigger weights are penalized more heavily than smaller ones. This results in the weights being more spread out, rather than relying on any subset of the weights.
  - **Handles multi-collinearity:** if any input features of x are highly correlated, the weights assigned to those features would be large, which results in unstable outputs (high variance). L2 Regularization shrinks the weights of any correlated features equally, leading to a more stable model.
  - **Overall more stability:** by keeping all the weights small and similar in scale, L2 helps a model behave more predictably on new data, since any noisy training examples cannot force the weights to change in magnitude by a lot.

Regularization Functions: L1

- *L1 Regularization:* $R(W) = \sum_{i,j} |W_{ij}|$
  - i.e. the sum of the absolute values of all the weights
  - **Enables feature selection:** L1 regularization shrinks some weights to 0, which means that only the most important features are considered in the final output of the model. This is a way to perform feature selection.
  - **Maintains interpretability:** due to the fact that some weights may be 0, this reduces the number of features used in the output, which makes it easier to interpret. Furthermore, since only the important features are used, we can clearly see which features actually affect our output.

Example

$$x = [1,1,1,1], w_1 = [1,0,0,0], w_2 = [0.25, 0.25, 0.25, 0.25]$$

which set of weights is better using **L2 regularization**?

First let us calculate the predictions in both cases (Both are identical)

$$x^T w_1 = 1 \times 1 + 0 \times 1 + 0 \times 1 + 0 \times 1 = 1$$
$$x^T w_2 = 0.25 \times 1 + 0.25 \times 1 + 0.25 \times 1 + 0.25 \times 1 = 1$$

Now calculate **L2 Regularization Penalty:**

$$R(w_1) = (1)^2 + (0)^2 + (0)^2 + (0)^2 = 1$$
$$so\ g(w) = data\ loss + 1$$
$$R(w_2) = (0.25)^2 + (0.25)^2 + (0.25)^2 + (0.25)^2 = 0.25$$
$$so\ g(w) = data\ loss + 0.25$$

w_2 is better

- Both weight sets make the same prediction
- But w_2 has a much lower regularization penalty (0.25 vs 1.0).
- L2 regularization prefers **spreading weight across all features** rather than concentrating it in one feature, which is exactly what w_2 does.

$$x = [1,1,1,1], w_1 = [1,0,0,0], w_2 = [0.25, 0.25, 0.25, 0.25]$$

which set of weights is better using **L1 regularization**?

$$R(w_1) = |1| + |0| + |0| + |0| = 1$$
$$so\ g(w) = data\ loss + 1$$
$$R(w_2) = |0.25| + |0.25| + |0.25| + |0.25| = 1$$
$$so\ g(w) = data\ loss + 1$$

w_1 is better

- Both weight sets make the same prediction and same penalty, BUT:
- L1 regularization **prefers sparsity** (fewer non-zero weights)
  - Feature selection: Only 1 feature is active
  - Interpretability: Simpler model
  - Efficiency: Fewer computations needed
- For this same example:
  - L2 prefers w_2 (spreads weights evenly)
  - L1 prefers w_1 (concentrates weights, creates sparsity)

Notes on Regularization

- Typically, L2 regularization is used more in practice due to improving stability and handling multi-collinearity.
- Another form of regularization called Elastic Net, which combines the penalties from L1 and L2 using separate constants λ to determine amount of regularization from each method:

$$\lambda_1 \sum_{i,j} |W_{ij}| + \lambda_2 \sum_{i,j} W_{ij}^2$$

- Note that in both examples from the two previous slides, $x^T w_{L1} = x^T w_{L2} = 1$. This is not always true (resulting output from using two different types of regularization is not necessarily the same), but in this case, it demonstrates how we express our preference for the behaviour of the model.
  - Assuming the features were correlated, L1 regularization reduced all others to 0 causing feature selection, while L2 regularization kept all weights small and similar in size leading to more stability.
  - So our choice of regularization function would depend on what qualities we preferred for the model to have!

Other Regularization Methods

- There are other methods of regularization that do not add a term to the cost function, such as Dropout and Batch Normalization, but these will be discussed in later lectures.